Análise e Desenvolvimento de Sistemas — Prof. Emerson Ribeiro de Mello
Segurança da Informação — mello@ifsc.edu.br

Prática: OWASP WebGoat e WebWolf

18/11/2025

Integrantes:
Giuliano Marcus Bianco
Luís Renato Freitas de Almeida
Nicolas Arthur Raulino Oliveira

1 Objetivos

- Configurar e iniciar o ambiente da prática usando Docker Compose
- Acessar as aplicações WebGoat e WebWolf
- Usar o OWASP ZAP para interceptar e analisar requisições HTTP
- Atacar algumas vulnerabilidades do WebGoat

2 Configuração do Ambiente

O ambiente de laboratório é composto por três aplicações principais:

- **WebGoat**: Aplicação web deliberadamente insegura que contém vulnerabilidades comumente encontradas em aplicações Java. É utilizada para aprender e praticar técnicas de segurança web através de exercícios práticos.
- **WebWolf**: Aplicação complementar que simula o ambiente do atacante. Permite receber e-mails, hospedar arquivos e capturar requisições enviadas pelo WebGoat durante os exercícios.
- **OWASP ZAP**: Proxy de segurança para interceptar e analisar requisições HTTP/HTTPS, permitindo identificar vulnerabilidades e testar ataques.

2.1 Arquivo docker-compose.yml

O arquivo de configuração do Docker Compose é apresentado no Listagem 1.

```yaml
services:
  webgoat:
    image: webgoat/webgoat:latest
    container_name: webgoat
    user: "root"
    restart: unless-stopped
    networks: [zapnet]
    ports:
      - "127.0.0.1:9090:9090"
    volumes:
      - webgoat_data:/home/webgoat/.webgoat-2025.3

  zap:
    image: zaproxy/zap-stable
```

```
    container_name: zap
    user: "zap"
    restart: unless-stopped
    networks: [zapnet]
    environment:
      DISPLAY: ${DISPLAY}
    volumes:
      - /tmp/.X11-unix:/tmp/.X11-unix
      - zap_data:/home/zap/.ZAP
    command: zap.sh

networks:
  zapnet:

volumes:
  zap_data:
  webgoat_data:
```

3 Iniciando o Ambiente

3.1 Iniciar as aplicações

Para iniciar o WebGoat, WebWolf e OWASP ZAP, execute o comando abaixo:

```
docker-compose up --build -d
```

4 Acessando as Aplicações

4.1 Acessando o OWASP ZAP

O OWASP ZAP possui interface gráfica que será exibida automaticamente após a inicialização. O ZAP atua como um *Man-in-the-Middle* (MITM), posicionando-se entre o navegador e o servidor web para interceptar, analisar e modificar o tráfego HTTP/HTTPS. A interface permite visualizar todas as requisições e respostas capturadas.

4.2 Acessando o WebGoat

O WebGoat é a aplicação principal desta prática. Ela contém diversas vulnerabilidades comumente encontradas em aplicações Java. Para acessar o WebGoat:

1. Abra a interface gráfica do OWASP ZAP
2. Use o navegador integrado do ZAP (disponível na interface)
3. No navegador integrado, acesse:

   ```
   http://webgoat:8080/WebGoat
   ```

O navegador integrado do ZAP já está configurado para interceptar automaticamente todo o tráfego HTTP/HTTPS, permitindo analisar as requisições e respostas enviadas pelo WebGoat.

> **! Importante**
>
> Use `webgoat:8080` ao invés de `localhost:8080` no navegador integrado do ZAP, pois as aplicações estão na mesma rede `zapnet` que foi definida no arquivo `docker-compose.yml`.

4.3 Acessando o WebWolf

O WebWolf simula o ambiente do atacante e é utilizado durante os exercícios para receber dados enviados pelo WebGoat. Diferente do WebGoat, o WebWolf deve ser acessado através do navegador na sua máquina local:

1. Abra seu navegador web
2. Acesse:

```
http://localhost:9090/WebWolf
```

O WebWolf permite visualizar e-mails recebidos, arquivos hospedados e requisições capturadas durante os exercícios práticos do WebGoat.

4.4 Iniciando com o WebGoat

Na página de Login do WebGoat em http://webgoat:8080/WebGoat no navegador do Zap, devemos seguir estes passos:

1. Criar uma conta clicando em `"or register yourself as a new user"`
2. Preencha as informações com dados fictícios para criação de um usuário. (Exemplo: Username: `"fulano"` Senha: `"123456"`)
3. Clique em `"Sign up"`
4. Dentro do ZAP, podemos encontrar o usuário e a senha em texto claro que criamos dentro dos pacotes no histórico de requisições.

```
http://webgoat:9090/WebWolf
```

5 Vulnerabilidades em JSON Web Tokens (JWT)

JWT é um padrão usado para autenticação em aplicações web. Estrutura: header.payload.signature, onde cada parte é codificada em base64.

5.1 Exercício 1: Decodificando um JWT

JWTs são apenas codificados em base64, não criptografados. Qualquer pessoa pode decodificar e visualizar o conteúdo.

1. Acesse: `(A7) Identity & Auth Failure > JWT tokens > Exercício 4`
2. Copie o token JWT fornecido
3. Acesse o WebWolf: `http://localhost:9090/WebWolf`
4. Na aba `"JWT"`, cole o token e clique em `"Encoded"`
5. No payload decodificado, localize o campo `"user_name": "user"`
6. Volte ao WebGoat e insira o username encontrado.

5.2 Exercício 2: Manipulação de JWT - Tornando-se Admin

Se a aplicação aceita tokens com algoritmo "none"(sem assinatura), é possível modificar o payload para elevar privilégios.

1. Acesse: `(A7) Identity & Auth Failure > JWT tokens > Exercício 6`
2. No ZAP History, localize requisições para `/WebGoat/JWT/` e copie o token JWT do header `Authorization`
3. No WebWolf (aba JWT), decodifique o token
4. Modifique o header para:

```
{ "alg": "none" }
```

5. Modifique o payload para:

```
{ "admin": "true"}
```

6. Gere o novo token (sem assinatura)
7. No ZAP Requester, substitua o token no header e envie

6 Injeção SQL

Injeção SQL é um ataque que explora vulnerabilidades em aplicações que interagem com bancos de dados, permitindo que um atacante execute comandos SQL não autorizados.

1. Dentro do WebGoat, vamos acessar `(A3) Injection > SQL Injection (intro) > Página 2`.
2. Em `"SQL Query"`, vamos colocar o código abaixo:

```
SELECT department FROM employees WHERE userid=96134;
```

3. Clicar em `"Submit"`.
4. Acessar o ZAP e procurar por uma requisição de método POST com Tags JSON e verificar se contém a nossa Query SQL.
5. Após identificarmos, podemos clicar em `"Requester"` na parte superior direita do ZAP e selecionar o Method POST.
6. Vamos colocar os mesmos dados do cabeçalho que o pacote com a query SQL que enviamos anteriormente pelo WebGoat.
7. Podemos clicar em `"Send"` e ver a resposta que o sistema nos fornece.
8. Se estiver correto, podemos esperar uma resposta assim:

```
{
  "lessonCompleted" : true,
  "feedback" : "You have succeeded!",
  "feedbackArgs" : null,
```

```
  "output" : "<span class='feedback-positive'>select department from employees where userid=
    96134<\\/span><table><tr><th>DEPARTMENT<\\/th><\\/tr><tr><td>Marketing<\\/td><\\/tr><\\/
    table>",
  "outputArgs" : null,
  "assignment" : "SqlInjectionLesson2",
  "attemptWasMade" : true
}
```


O objetivo deste ataque foi personificar o sistema para conseguir informações confidenciais de um banco de dados.

Documento licenciado sob Creative Commons “Atribuição 4.0 Internacional”.